TFC

Víctor Pérez Quesada

2ºDAW - 2024

OBJETIVO

Desarrollo aplicación web marketplace de productos locales con Django.

JUSTIFICACIÓN

La evolución del mercado con Internet, ha sido tal que se ha permitido comprar productos a nivel global. No obstante, al mismo tiempo que se ha permitido comprar productos fácilmente de cualquier parte del mundo, se ha descuidado la venta local.

Es decir: ¿por qué es más fácil buscar desde tu casa productos que están en otro país que buscar aquello que se vende en la tienda de al lado?

La **idea** es crear una web de búsqueda de productos parecido al de google shopping pero no estará destinado a vender productos online.

Hay otras parecidas pero son una absoluta basura:

- Provip: Ni siquiera tiene página web
- Aprop: Es un marketplace que venden como tal

https://hipertextual.com/2021/11/fiebre-marketplaces-comercio-local
Porque el planteamiento que realizaron no tiene ningún sentido.

- No se trata de crear una web funcional solo en una ciudad
- No se trata de crear un "marketplace" donde las tiendas vendan a través de la web
- Y mucho menos se trata de que un gobierno municipal cree un marketplace

Nuestra propuesta es la siguiente:

- Una web para todo el mundo, donde se selecciona por país, ciudad, barrio, geolocalización…
- Los productos se añaden vinculados a una localización física
- Los productos no se venden en la app, ni por internet, sino en la localización física
- Los resultados de las búsquedas se posicionarán en Google aunque no se venda a través de internet, de esa forma se promociona la web

Obtención de ingresos:

Mediante la visibilidad preferencial de vendedores suscriptores.

Identidad Corporativa:

<u>Marca:</u>

Local Market

Imagen de marca:

Análisis de la competencia

Diferencias con otros competidores:

- Wallapop y similares: venta de segunda mano
- Amazon y similares: venta online
- Google Shopping: También muestra las opciones online. No permite cambiar la localización. No permite filtrar productos por venta local.
- Google Maps: La interfaz es horrible, no muestra bien los resultados de las tiendas y no se puede buscar por productos específicos.

¿Se ha intentado?:

- Provip: No tiene página web.
- Aprop: No tiene plataforma online accesible con los productos

Beneficios para los clientes compradores:

- Poder buscar productos específicos que se venden en su ciudad
- Evitar pagar los envíos
- Evitar tener que esperar a qué envíen el pedido

Beneficios para los clientes vendedores:

- Pueden promocionarse
- Pueden aprovechar ciertos beneficios de la venta online sin sus costes.
- Puede tener una entrada a la venta online más fácil. El coste de entrada es menor, no necesitan web ni deberle comisiones o responsabilidades a Amazon, no deben comprometerse a vender online, es para atraer clientes. No obstante, luego podrían empezar a aceptar realizar envíos.

Análisis DAFO:

- Debilidades:

El valor de la app aumenta cuantas más personas suban sus productos. Por ello, al principio hay un gran obstáculo por no tener clientes.

- Amenazas:

Google Shopping o otras apps pueden actualizarse con menor dificultad para hacer lo mismo que nosotros.

- Fortalezas:

Coste de negocio muy bajo.

- Oportunidades:

Gente que no quiere aprender a usar internet.
Mercado negro.

Cinco Fuerzas De Porter:

- Entrada de nuevos competidores:

Alta. Competidores globales. Dificultad de implementación de la app relativamente fácil.

- Rivalidad entre competidores existentes:

  Alta, podrían adaptarse fácilmente para incluir estas funcionalidades. Como Instagram con Snapchat.

- Amenaza de productos sustitutivos:

Riesgo de que la gente prefiera siempre comprar online.

- Poder de negociación del proveedor:

  Medio, no se necesitan muchos proveedores en este negocio.

- Poder de negociación del cliente:

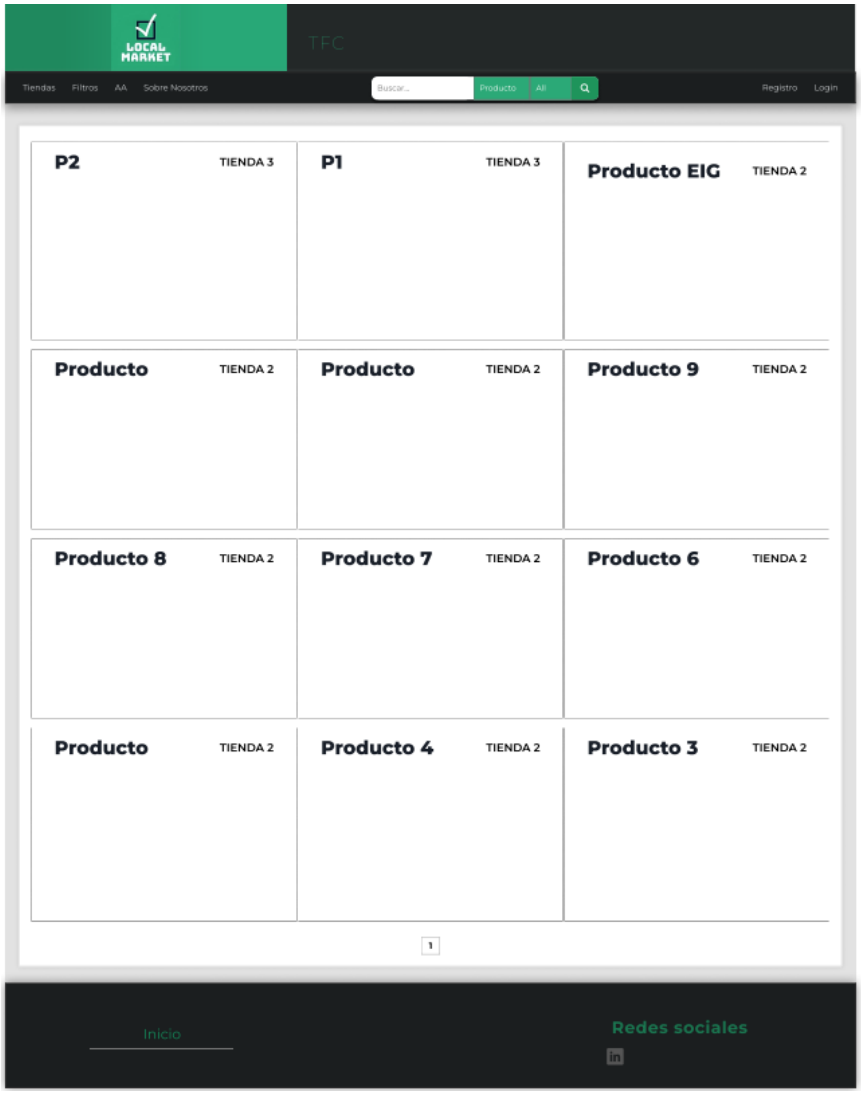
Alto, puede prescindir fácilmente de la app o irse a otra app.

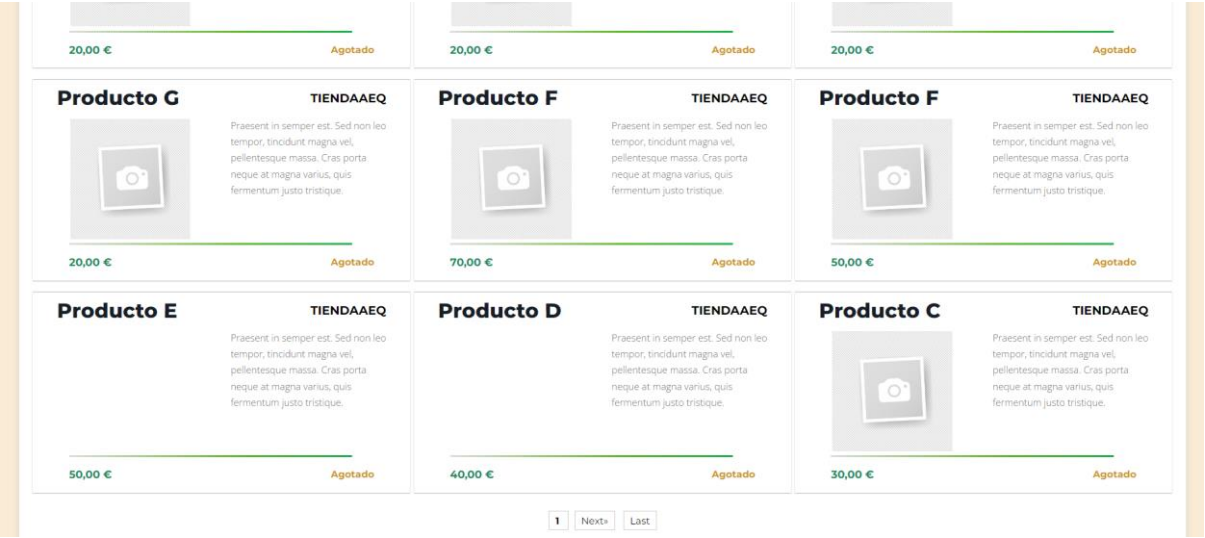
Análisis PESTEL:

- Política: +impuestos, +gasto público, +deuda
- Económica: Seguridad Social insostenible. Inflación. Subida tipos de interés para contener inflación. Paro alto. Deuda 2 billones.
- Social: Más ventas por internet. Trabajo a distancia.

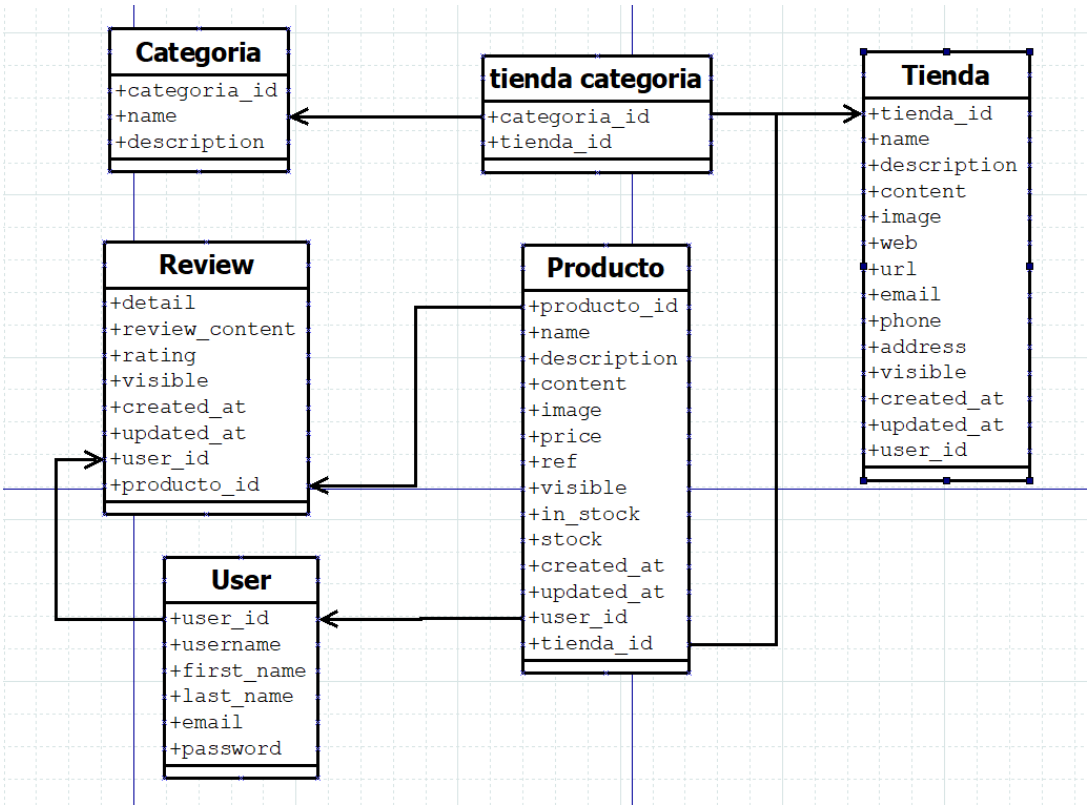
- Tecnológica: Avanzan muy rápido las tecnologías relacionadas con el desarrollo de apps. Tecnologías más rápidas (+productividad, +competencia).
- Ecológico: Comprar en persona reduce la contaminación por transporte.
- Legal: Protección de datos.

Diseños de la Aplicación

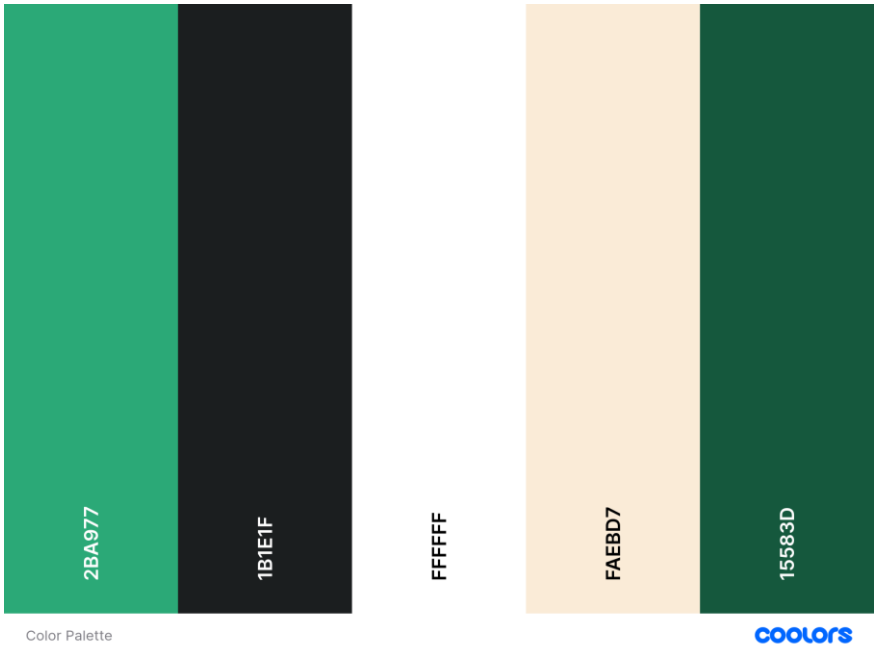




Diseño de la estructura de datos:



Guía de estilos:



Fuente:

[Montserrat - Google Fonts]

Experiencia de Usuario

Usuario guest (sin login):

- Ver tiendas -> Página de tienda -> Ver producto
- Registrarse / Login

Usuario además puede:

- Añadir reviews en página de producto
- Mi perfil ->
  - Editar información de perfil.
  - Editar y borrar reviews.

Usuario Owner además puede:

- Mi perfil ->
  - Crear tienda
  - Mis tiendas
- Acceder a editar tienda en página de tienda o en Mi Perfil
- Editar tienda ->
  - Cambiar información
  - Crear productos de la tienda
  - Importar productos de la tienda a partir de un archivo .csv
- Vista de producto ->
  - Editar y eliminar producto.

Administrador además puede:

- Acceder al panel de administración:
  http://127.0.0.1:8000/admin/
- Ver, crear, editar y eliminar desde el panel de administración los registros incluyendo usuarios y grupos

Código

Django

1. Instalar Django

```
pip install Django
```

```
python -m django –version
```

La versión usada es la 5.0.2

2. Crear Proyecto

```
django-admin startproject nombreProyecto
```

```
python manage.py migrate
```

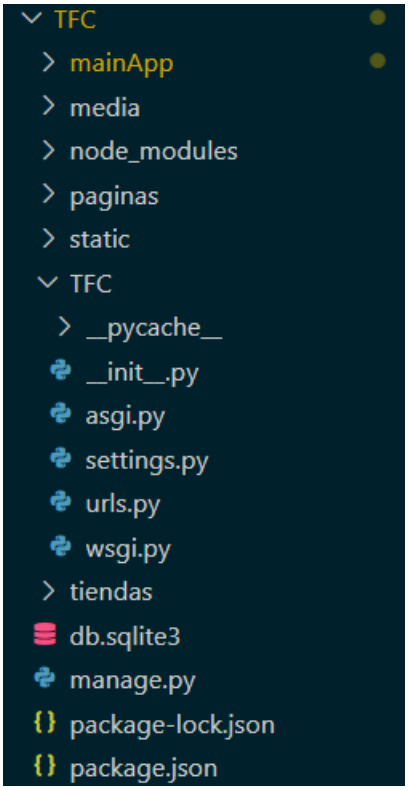
Generar una base de datos con funcionalidades por defecto como el user.

```
python manage.py runserver
```

Ejecutar el servidor en localhost: 127.0.0.1:8000

Los desarrollos en django se clasifican en apps, que son paquetes con funcionalidades contenidas y reutilizables.

Esta es la organización de las carpetas del proyecto:



La carpeta TFC > TFC, se crea automaticamente y gestiona los settings.py de todo el proyecto y las urls.py principales.

Tenemos 3 apps:

mainApp -> Características generales del proyecto:

- Carpeta static para css e imágenes
- Templates prinpales como layout.html
- login y registro

paginas -> generador de páginas estáticas a través del panel de administración

tiendas -> todo lo relativo a las tiendas y productos

Para crear una app:

```
python manage.py startapp myApp
```

Django usa el MVT (Model – View - Template).
Los modelos se guardan en models.py, que contiene las clases cons las estructuras de los objetos.
La vista es el archivo views.py que guarda los métodos que ejecutan la lógica y llaman a los templates.
Los templates son los archivos.html que muestan la información al usuario.

(Comparando con otros frameworks con el diseño MVC o Modelo-Vista-Controlador: Las apps de Django serían los controladores del MVC. Las vistas de Django serían las acciones del MVC. Los templates de Django serían las vistas del MVC).

Reconocer código de Django en editor:

```
pip install pylint-django
```

Añadir la app a INSTALLED_APPS en settings.py:

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'mainApp',
```

Rutas:

```
urlpatterns = [
    path('', views.getTiendas, name="index"),
    path('tiendas/', views.getTiendas, name="tiendas"),
    path('tienda/<int:tienda_id>', views.getTienda, name="tienda"),
```

Las rutas tienen que estar en el archivo urls.py de la carpeta principal del proyecto (TFC), pero se pueden importar las urls de los archivos urls.py de otras apps.

```
urlpatterns = [
      path('', include('paginas.urls')),
```

Views:

Las rutas llaman a la vista. Se llaman con el valor en “name” y llama al método “views.metodo” con los parámetros pasados por la url si los tiene “tienda/<str:param>”.

Renderizar vista:

```
def index(request):
    return render(request, 'mainApp/index.html', {'title': 'Inicio'})
```

Renderiza el template pasando las variables del contexto: {“nombreVariable”: “valorVariable”}

Templates:

Se puede crear un layout principal para utilizar como plantilla del resto de templates:

En layout.html:

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
    <title>{%block title%}{%endblock%}</title>

</head>
<body>
    <header></header>
    <main>
        <div id="content">
            <div class="box">
                {% block content %}
                {% endblock %}
            </div>
        </div>
    </main>
```

Utlizar layout en otros templates:

```
{% extends 'layouts/layout.html' %}
{% block title %} {{title}} {% endblock %}
{% block content %}
      <section> Contenido </section>
{% endblock %}
```

Se puede mantener contenido del layout que estuviera dentro del elemento block con: {{ block.super }}

CSS:

Crear carpeta 'static'.

Cargar carpeta en layout: `{% load static %}`

Aplicar estilos: `<link rel="stylesheet" type="text/css" href="{% static 'css/styles.css' %}">`

Comentarios en templates:

```
{#  #}
{% comment "" %}{% endcomment %}
```

Llamar variable en el template recibida desde view: `{{mi_variable}}`

Condicionales:

```
{% if my_variable and my_variable != 'null' %}
      <p>{{my_variable}}</p>
      {% else %}
```

```
      <p>El nombre no existe</p>
{% endif %}
```

Bucles:

```
{% for leng in coding %}
      <li>{{leng}} {{forloop.counter}}</li>
{% empty %}
      <p>This category is empty</p>
{% endfor %}
```

Bucle iterando 5 veces:

```
{% for i in '12345'|make_list %}
    {{ forloop.counter }}
{% endfor %}
```

Pasar String a Int en template para comparación

```
      i|add:"0"
```

```
{% for i in '12345' %}
    {% if i|add:"0" <= review.rating %}
        <span class="star"><img src="{% static 'img/star_fill.png' %}"
alt="Filled star"></span>
    {% else %}
        <span class="star"><img src="{% static 'img/star_empty.png' %}"
alt="Empty star"></span>
    {% endif %}
{% endfor %}
```

Incluir otros templates:

```
{% include 'curr_date.html' %}
```

Usar variables al incluir otros templates:

```
{% include 'curr_date.html' with greet="Hey! " %}
```

Llamar a la vista con enlaces (se usa el name del path):

```
<a href="{% url 'goodnight_world' %}">Goodnight World</a>
```

Fecha:

```
{% now "d/m/Y" %}
```

Filtros:

```
{{"Quitar ñ ñññ"|cut:"ñ"}}
{{empty_variable|default:"default text if empty"}}
{{first_element|first}}
{{last_element|last}}
{{join_elements|join:', '}}
```

```
{{variable|length}}
{{array|random}}
{{"Contar palabras"|wordcount}}

{{"<h2>No html tags</h2>"|striptags}}
```

Autoescape: Cross-Site Scripting (XSS)

```
{{% autoescape on %}} {{variable}} {{% endautoescape %}}
```

Interpretar HTML:

```
{{"<h1>Interpretar HTML</h1>"|safe}}
```

Filtros personalizados:

Crear filters.py

```
from django import template
register = template.Library()
@register.filter(name='hey')
def filtroPersonalizado(value):
error = ''
if(len(value)>=8):
      error = 'Your name is too long'
      return error


{% load filters %}
{{"Filter personalizado"|filtroPerosna}}
```

MODELOS:

```
class Article(models.Model):
title = models.CharField(max_length=100)
content = models.TextField()
image = models.ImageField(default='null')
public = models.BooleanField()
created_at = models.DateTimeField(auto_now_add=True), # auto_now_add crea
registro solo la primera vez
updated_at = models.DateTimeField(auto_now=True)
```

El código puede pasar a base de datos directamente:

```
python manage.py makemigrations
```

Esto añade 0001_initial.py. El id lo crea automáticamente.

Ahora, crear tablas en sql.

```
python manage.py sqlmigrate myApp 0001
```

Aplicar cambios.

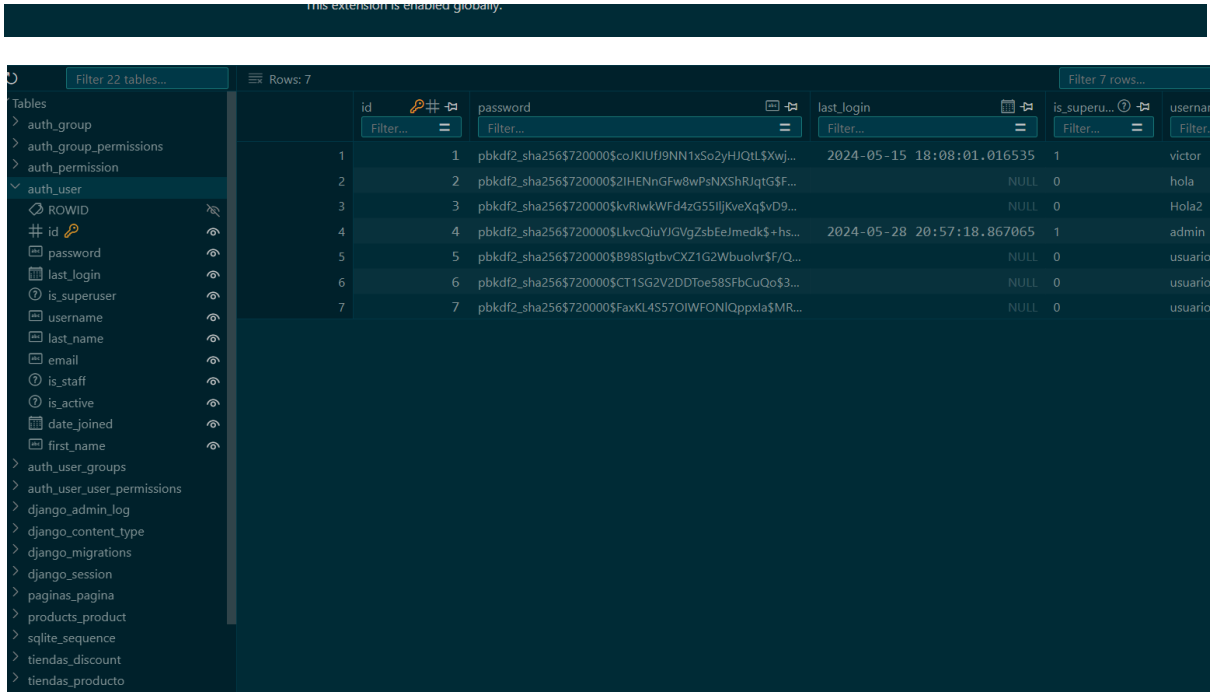
```
python manage.py migrate
```

```
python manage.py makemigrations
python manage.py sqlmigrate myApp 0002
python manage.py migrate
```

Importar en vista si se quiere usar para acceder a BD

```
from myApp.models import Article, Category
```

Ver base de datos con extensión SQLite Viewer de VSCode



INTERACCIÓN CON BD. ORM:

```
def create_article(request):
articulo = Article(
      title = 'First Article',
      content = 'This is the content of the article',
      public = True
)
articulo.save()

def article(request):
try:
      article = Article.objects.get(pk=1, public=True)
      response = f"Articles: {article.title}"
except:
      response = "0 Articles Found"
return HttpResponse(response)

def articles(request):
try:
```

```
        articles = Article.objects.all()
        params = {'articles': articles}
except:
        params = {'articles': "0 Articles Found"}
return render(request, 'articles.html', params)
```

Ordenar, limitar:

```
articles = Article.objects.order_by('id')
articles = Article.objects.order_by('-id')[:3]
```

Buscar:

```
articles = Article.objects.filter(title="Batman", Public=True)
articles = Article.objects.filter(title="Batman").exclude(public=False)
articles = Article.objects.filter(title__contains="Batman")
articles = Article.objects.filter(title__iexact="Batman")
articles = Article.objects.filter(id__gt=10)
```

Borrar:

```
def delete_article(request, id):
try:
        article = Article.objects.get(pk=id)
        article.delete()
        params = {'articles': articles}
except:
        params = {'articles': "0 Articles Found"}
return render(request, 'articles.html', params)
```

Cambiar fecha automática de created_at:

settings.py -> LANGUAGE_CODE

Realizar Consulta sin ORM:

```
articles = Article.objects.raw("SELECT id, title FROM myApp_article WHERE
title = 'Article1' AND public=True")
```

Realizar Consulta OR:

```
from django.db.models import Q # filtrar

article = Articles.filter(
    Q(title__contains="2") | Q(title_contains="3")
)
```

Acceso a tablas relacionadas

En vista -> ORM

```
producto = Producto.objects.select_related('tienda').get(id=producto_id)
reviews = Review.objects.filter(producto_id=id).select_related('user')
```

Acceder al valor de la variable relacionada

```
<span class="name">{{producto.user.lastname}}</span>
```

Error 404

```
tienda = get_object_or_404(Tienda, id=tienda_id)
```

Paginación

```
from django.core.paginator import Paginator
pagination = Paginator(tiendas, 10)
tiendasPorPagina = pagination.get_page(page)
```

Acceder a las distintas páginas:

```
<div class="paginator">
    {{ results.has_previous }}
    {{ results.previous_page_number }}">
    {{ results.number }}
    {{ results.has_next }}
    {{ results.next_page_number }}
    {{ results.paginator.num_pages }}
</div>
```

FORMULARIOS:

Tomar datos recibidos de formulario en la vista:

```
if request.method == 'POST':
title = request.POST['title']
```

Protección contra CSRF:

```
<form action="" method="">
{% csrf_token %}

</form>
```

Clases Formularios:

```
from django import forms
from django.core import validators
from django.contrib.auth.forms import UserCreationForm
from django.contrib.auth.models import User
```

```
class RegisterForm(UserCreationForm):
    # crear formulario a partir de un modelo
    class Meta:
        model = User
        fields = ['username', 'email', 'first_name', 'last_name',
'password1', 'password2'] # campos del objeto auth_user de django
```

Usar formulario en views

```
from .forms import RegisterForm

def registerView(request):
    registerForm = RegisterForm()
    if request.method == 'POST': # si se envía el formulario
        registerForm = RegisterForm(request.POST)
```

Usar formulario en templates

```
    {{registerForm.errors}}
    <form method="post" action="">
        {% csrf_token %}
        {% for field in registerForm %}
            {{field.label}}
            {{field}}
        {% endfor %}
        <input type="submit" value="Crear Usuario"/>
    </form>
```

Mensaje de éxito y error en formulario

```
from django.contrib import messages

messages.success(request, 'Has creado un Usuario con éxito')
```

Asignar atributos como clases a los inputs de los formularios Django

```
class ProductoForm(forms.ModelForm):
    class Meta:
        model = Producto
        fields = ["name", "ref", "description", "content", "image",
"price", "in_stock", "visible"]

    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
```

```
        self.fields['name'].widget.attrs.update({'class': 'form-control'})
```

Crear validación para formulario Django

```
class ProductoForm(forms.ModelForm):
    class Meta:
        model = Producto
        fields = ["name", "ref", "description", "content", "image",
"price", "in_stock", "visible"]

    def clean_price(self):
        price = self.cleaned_data.get('price')
        min_price = 0.00
        try:
            price = float(price)
        except (TypeError, ValueError):
            raise forms.ValidationError('El precio no es un valor
válido.')
        if price < min_price:
            raise forms.ValidationError(f'El precio debe ser al menos
{min_price}.')
        return price
```

Usar un select en un formulario

```
class ReviewForm(forms.ModelForm):
    class Meta:
        model = Review
        fields = ["rating", "review_content"]

    RATING_CHOICES = [(i, i) for i in range(1, 6)]
    rating = forms.ChoiceField(
        choices=RATING_CHOICES,
        widget=forms.Select(attrs={'class': 'form-control'}),
        required=False
    )
```

Autenticación

```
from django.contrib.auth import authenticate, login, logout
```

Bloquear rutas sin estar autenticado

```
from django.contrib.auth.decorators import login_required
```

```
@login_required(login_url="login")
def method(request):
```

```
if request.user.is_authenticated:
```

Comprobar si usuario es el dueño del registro

```
if tienda.user_id == request.user.id:
```

Redireccionar:

```
return redirect('login')
```

Redireccionar a la una url usando parametros de url:

```
tienda_url = reverse('tienda', kwargs={'tienda_id': tienda_id})
return redirect(tienda_url)
```

Este devuelve a tienda/<int:tienda_id>

Redireccionar sin volver a enviar el formulario:

```
producto_url = reverse('producto', args=[producto.pk])
                return HttpResponseRedirect(producto_url)
```

ADMIN:

Crear usuario administrador:

```
pyhon manage.py createsuperuser
```

Añadir elementos a panel de administración:

```
admin.py -> admin.site.register(Product)
```

Cambiar nombre de modelos en el panel de administración:

```
apps.py
```

```
class ProductsConfig(AppConfig):
    default_auto_field = 'django.db.models.BigAutoField'
    name = 'products'
    verbose_name = "Gestión de Productos"
```

```
Cambiar INSTALLED_APPS en settings.py:
```

```
'products.apps.ProductsConfig'
```

Cambiar título

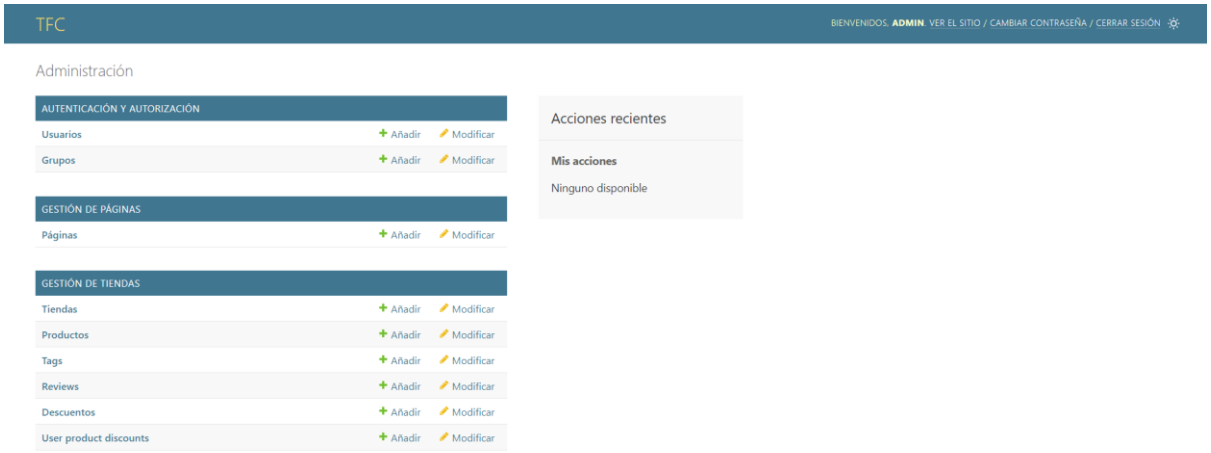
```
title = "TFC"
```

```
subtitle = "Administración"
admin.site.site_header = title
admin.site.site_title = title
admin.site.index_title = subtitle
```

Personalizar panel de administración

```
class PaginaAdmin(admin.ModelAdmin):
    # configurar campos solo lectura
    readonly_fields = ('create_at', 'updated_at')
    # añadir buscador
    search_fields = ('title',) # , aunque solo sea uno para que sea tupla
    # filtrar por propiedad
    list_filter = ('visible',)
    # mostrar campos
    list_display = ('title', 'visible')
    # ordenar
    ordering = ('-created_at')

class TiendaAdmin(admin.ModelAdmin):
    # buscar por tablas relacionadas
    search_fields = ('name','user__username','tags__name')
```



Context Proccessor

En settings.py -> TEMPLATES Usar en todas las templates, independientemente de la app.

Editor de texto enriquecido. CKEDITOR 5

https://pypi.org/project/django-ckeditor-5/
pip install django-ckeditor-5

Instalar app en settings.py (hay que volver a hacer python manage.py runserver)

```
INSTALLED_APPS = [
      'django_ckeditor_5'
```

Añadir configuración a settings.py:
https://pypi.org/project/django-ckeditor-5/

Añadir en urls:

```
path('ckeditor5/', include('django_ckeditor_5.urls')),
```

Utilizar en models.py

```
from django_ckeditor_5.fields import CKEditor5Field

content = CKEditor5Field(verbose_name='Contenido', null=True, blank=True,
config_name='extends')
```

Interpretar correctamente etiquetas HTML

```
{{pagina.content|safe}}
```

Personalizar toolbar:

```
CKEDITOR_5_CONFIGS = {
    'default': {
        'toolbar': ['heading', '|', 'bold', 'italic', 'blockQuote', '|',
'link', 'bulletedList', 'numberedList', '|', 'outdent', 'indent',
'alignment', '|', 'fontSize', 'fontColor', 'fontFamily', '|',
'imageInsert', 'insertTable', 'mediaEmbed', '|', 'undo', 'redo' ],
    },
```

Utilizar en front:
Por ahora funciona en el panel de admin. Para utilizarlo en el front con un toolbar personalizado:

1. Crear un build en: https://ckeditor.com/ckeditor-5/online-builder/
2. Subir los archivo 'ckeditor.js' y 'styles.css' de las carpetas del builder descargadas a las carpetas estáticas del proyecto.
3. Importar en layout:

```
{% block scripts %}
    <script src="{% static 'js/ckeditor.js' %}"></script>
```

```
    <script  src="{% static 'js/customCKeditor.js' %}"></script>
{% endblock %}
```

4. Utilizar en el template. Se aplican las configuraciones y los estilos al crear un formulario Django de un modelo con un campo CKeditor:

```
<form method="post" action="" enctype="multipart/form-data">
      {% csrf_token %}
      {% for field in tiendaForm %}
      <div class="form-group">
           <label>{{ field.label_tag }}</label>
                {{ field }}
      </div>
      {% endfor %}
      <input type="submit" value="Crear Tienda"/>
</form>
```

El usuario puede crear páginas personalizadas que se mostraran en la vista de su tienda.
Formulario en CKeditor:

Resultado:

SASS en Django

```
pip install django-compressor
pip install django-libsass
```

settings.py

```
# SASS ____________________________________________________________
STATIC_ROOT = os.path.join(BASE_DIR, 'static/')
STATICFILES_FINDERS =(
'django.contrib.staticfiles.finders.FileSystemFinder',
'django.contrib.staticfiles.finders.AppDirectoriesFinder',
'compressor.finders.CompressorFinder',
)
COMPRESS_PRECOMPILERS = (
    ('text/x-scss', 'django_libsass.SassCompiler'),
)
```

layout.html

```
    {% load static %}
```

```
    {% load compress %}
    {% compress css %}
        <link rel="stylesheet" type="text/x-scss" href="{% static
'css/styles.scss' %}">
    {% endcompress %}
```

Es necesario cargar static (`{% load static %}`) en cada template que se vaya a usar.

Importar productos a partir de un CSV

Las columnas que puede tener el csv son:
ref (number, unique), name (string), description (string), price (number, decimal), visible (boolean), in_stock (boolean)

222	P2	P2 cont	200	true	false

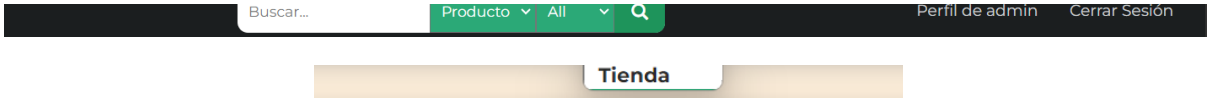
```
import csv
import io
from django.db import IntegrityError
                # Example usage:
                # csv_file_path = 'path/to/your/csv/file.csv'
                # create_records_csv(csv_file_path)
                # return getTienda(request, tienda_id)

@login_required(login_url="login")
def importCSV(request, tienda_id):
    if request.user.is_authenticated:
        tienda = Tienda.objects.get(id=tienda_id)
        if tienda.user_id == request.user.id: # comprobar que el usuario
es el owner de la tienda
            # leer cada fila del csv y guardar el objeto
            if request.method == 'POST' and request.FILES.get('csv_file'):
                csv_file = request.FILES['csv_file']
                csv_data = csv_file.read().decode('utf-8') # solucionar
error: iterator should return strings, not bytes
                delimiter = ';' if ';' in csv_data.split('\n')[0] else ','
# delimiter en .csv puede ser ',' or ';'. Tomar primera linea y comprobar
si se encuentra ';'
                reader = csv.DictReader(io.StringIO(csv_data),
delimiter=delimiter) # https://www.youtube.com/watch?v=t3BdM6JlAmY.
DictReader() returns dictionary. reader() returns array.
                for row in reader:
                    row_ref = row.get('ref', 'default_value')
                    row_ref = row['ref']
```

```
                    productos_ref = Producto.objects.filter(ref=row_ref,
tienda_id=tienda_id)
                    if not productos_ref.exists(): # comprueba si existe o
no un producto en la tienda con la referencia. La referencia será el 'id
común' entre la app y el .csv.
                        # Si no existe. Se crea uno nuevo:
                        new_producto = Producto(
                            name=row['name'],
                            description=row['description'],
                            price=row['price'],
                            visible=row['visible'].lower() == 'true', #
condicional devuelve true si en cvs el valor es true case insensitive
                            in_stock=row['in_stock'].lower() == 'true'
                        )
                        new_producto.tienda_id = tienda_id
                        new_producto.user_id = request.user.id
                        new_producto.ref=row_ref
                        try:
                            new_producto.save()
                        except IntegrityError: # comprueba que no se
repitan elementos únicos como ref
                            pass
                    else: # si existe, seleccionar el producto y
actualizarlo
                        producto_ref = productos_ref.first() # primer
elemento de la query
                        producto_ref.name = row['name']
                        producto_ref.content = row['description']
                        producto_ref.price = row['price']
                        producto_ref.visible = row['visible'].lower() ==
'true'
                        producto_ref.in_stock = row['in_stock'].lower() ==
'true'
                        producto_ref.save()
                tienda_url = reverse('tienda', kwargs={'tienda_id':
tienda_id})
                return HttpResponseRedirect(tienda_url)
            else:
                tienda_url = reverse('tienda', kwargs={'tienda_id':
tienda_id})
                return HttpResponseRedirect(tienda_url)
        else:
```

```
            tienda_url = reverse('tienda', kwargs={'tienda_id':
tienda_id})
            return HttpResponseRedirect(tienda_url)
    else:
        return redirect('login')
```

Buscador



El buscador permite seleccionar si buscar en productos o tiendas y la categoría que se quiere buscar, todas por defecto. Al realizar una búsqueda la configuración se mantiene igual.

```
def search(request):
    model = request.GET.get('model', 'tienda').strip()
    query = request.GET.get('query', 'all').strip()
    query = query.replace(' ', '') # asegurarse de que no se envíen
espacios en blanco
    category = request.GET.get('category', 'all').strip()
    model_class = None
    if model == 'tienda':
        if query == 'all':
            return getTiendas(request)
        else:
            model_class = Tienda
    elif model == 'producto':
        model_class = Producto
    else:
        return getTiendas(request)
    if category != 'all':
        results = model_class.objects.filter(name__icontains=query,
tags__name=category) # name__icontains para tomar elementos que contengan
la query
    else:
        results = model_class.objects.filter(name__icontains=query)
    resultsPerPage = 12
    pagination = Paginator(results, resultsPerPage)
    paginatorPage = request.GET.get('page')
    resultsInPage = pagination.get_page(paginatorPage) #
    return render(request, 'search.html', {
        'title': 'Resultados Encontrados',
        'tiendas': resultsInPage,
        'productos': resultsInPage,
```

```
        'model': model,
        'query': query,
        'category': category
    })
```

Layout

```
<form method="get" action="{% url 'search' %}" >
    {% csrf_token %}
    <input id="query" name="query" type="text" placeholder="Buscar..." {%
if query and query != '' %}value="{{ query }}"{% endif %} />
    <select id="model" name="model">
        <option value="producto" {% if model == 'producto' %}selected{%
endif %}>Producto</option>
        <option value="tienda" {% if model == 'tienda' %}selected{% endif
%}>Tienda</option>
    </select>
    <select id="category" name="category">
        <option value="all" {% if category == 'all' %}selected{% endif
%}>All</option>
        {% for tag in tags %}
            <option value="{{tag.1}}" {% if category == tag.1 %}selected{%
endif %}>{{tag.1}}</option>
        {% endfor %}
    </select>
    <input class="websymbols" type="submit" id="search-submit"
name="search-submit" value="L"/>
</form>
```

Paginación de resultados

```
<div class="paginator">
    {% if tiendas.has_previous %}
    <a href="?page=1&model={{ model }}&query={{ query }}&category={{
category }}">First</a>
    <a href="?page={{ tiendas.previous_page_number }}&model={{ model
}}&query={{ query }}&category={{ category }}">&laquo;Anterior</a> <!--
página anterior -->
    {% endif %}
    <span class="actual">
        {{ tiendas.number }}
    </span>
    {% if tiendas.has_next %} <!-- si la pagina es > 1 -->
```

```
        <a href="?page={{ tiendas.next_page_number }}&model={{ model
}}&query={{ query }}&category={{ category }}">Next&raquo;</a>
        <a href="?page={{ tiendas.paginator.num_pages }}&model={{ model
}}&query={{ query }}&category={{ category }}">Last</a>
    {% endif %}
</div>
```

Bibliografía:

Django documentation | Django documentation | Django (djangoproject.com)

https://medium.com/@jm_rodrigues/how-to-create-custom-base64uploadadapter-ckeditor-e91332b89b0d

https://github.com/django-webpack/django-webpack-loader

https://ckeditor.com/ckeditor-5/online-builder/

https://ckeditor.com/docs/ckeditor5/

https://www.flaticon.com/free-icons/shapes-and-symbols

https://forum.djangoproject.com/t/looping-through-a-list-of-querysets-in-template/24753

https://stackoverflow.com/questions/4056883/when-should-i-use-escape-and-safe-in-djangos-template-system

https://forum.djangoproject.com/t/django-ckeditor-5-custom-media-storage-location-still-need-help/15064

https://stackoverflow.com/questions/77682219/ckeditor5-image-upload-issue-in-django-django-ckeditor-5

https://medium.com/@taylorberukoff/using-scss-sass-with-django3-d0930936610c

https://stackoverflow.com/questions/21938028/how-to-get-a-favicon-to-show-up-in-my-django-app

https://stackoverflow.com/questions/12370582/django-booleanfield-always-checked-even-when-value-is-false

https://forum.djangoproject.com/t/check-form-errors-if-is-valid-method-is-false/25787/2

https://realpython.com/customize-django-admin-python/

https://books.agiliq.com/projects/django-admin-cookbook/en/latest/set_ordering.html

https://stackoverflow.com/questions/72603256/update-or-create-from-file-csv-in-django

https://www.w3schools.com/django/ref_tags_filter.php

https://stackoverflow.com/questions/65565093/styling-django-filter-form-in-html

https://www.w3schools.com/django/django_update_record.php

https://docs.djangoproject.com/en/5.0/topics/forms/modelforms/

https://ordinarycoders.com/blog/article/django-modelforms